

ELTE EÖTVÖS LORÁND UNIVERSITY

ADVANCED SIGNALR

Modern web application development in .NET

Lecture 12

Dr. Máté Cserép

mcserep@inf.elte.hu

https://mcserep.web.elte.hu/

GROUP MANAGEMENT IN HUBS

Groups in SignalR allows to target a subset of connected clients.

- Each connection can belong to zero or more groups.
- Groups are dynamic: you can add or remove connections at runtime.
- Typical use cases include:
  - Private chat rooms
  - Team collaboration workspaces
  - Broadcasting updates to selected users, e.g. viewing the same user interface of the application

GROUP MANAGEMENT IN HUBS

Groups are just labels in the Hub, they don't have to be created explicitly

- Users sharing the same group label are in the same group
- Add a client to a group:

```
await Groups.AddToGroupAsync(
  Context.ConnectionId, "mygroup");
```

- Remove a client to a group:

```
await Groups.RemoveFromGroupAsync(
  Context.ConnectionId, "mygroup");
```

- Send message to a group of clients:

```
await Clients.Group("mygroup").SendAsync(
  "ReceiveMessage", "Server", "Hello Group!");
```

GROUP MANAGEMENT IN HUBS

- Send messages to a group of clients in a weakly-typed hub:

```
public class ChatHub : Hub
{
  public async Task SendMessage(
    string group, string user, string message)
  {
    await Clients.Group(group).SendAsync(
      "ReceiveMessage", user, message);
  }
}
```

GROUP MANAGEMENT IN HUBS

- Send messages to a group of clients in a strongly-typed hub:

```
public interface IChatClient {
  Task ReceiveMessage(string user, string message);
}
public class ChatHub : Hub<IChatClient>
{
  public async Task SendMessage(
    string group, string user, string message)
  {
    await Clients.Group(group)
                 .ReceiveMessage(user, message);
  }
}
```

GROUP MANAGEMENT IN HUBS

Further methods for finer control over group management:

- **`Clients.Group()`**: calls method on all connections in the group
- **`Clients.Groups()`**: calls method on multiple groups of connections
- **`Clients.GroupExcept()`**: calls a method on all connections in the specified group, except the specified clients
- **`Clients.OthersInGroup()`**: calls a method on a group of connections, excluding the client that invoked the hub method

AUTHENTICATION IN SIGNALR

SignalR integrates seamlessly with ASP.NET Core authentication middleware, which we have already used for WebAPI development.

- Clients must send a valid access token or cookie when connecting
- Token can be transparent (e.g. a JWT) or opaque as well
- The token can be transported as a *cookie*, in the *query string* or in a *HTTP header* (*Authorization* conventionally)
    - The use of Authorization header is only supported if the SignalR client is not running in a browser.
    - In case a WebSocket or SSE connection is initiated from a browser, it does not allow the use of an *Authorization* header.
    - The preferred solution is to include it in the *query string*.

AUTHENTICATION IN SIGNALR

- Cookie-based authentication works out of the box
  - Same origin is a requirement, otherwise CORS issues will arise
- JWT based authentication from query string requires configuration:

```
builder.Services.AddJwtBearer(options => {
  // ...
  options.Events = new JwtBearerEvents
  {
    OnMessageReceived = context => {
      context.Token = context.Request.Query["access_token"];
      return Task.CompletedTask;
    }
  };
});
```

AUTHENTICATION IN SIGNALR

- Custom (non-JWT) token authentication can be implemented by defining a custom **AuthenticationHandler**:

```
public class SimpleTokenHandler :
  AuthenticationHandler<AuthenticationSchemeOptions>
{
  // ... ctor ...
  protected override async
    Task<AuthenticateResult> HandleAuthenticateAsync()
  {
      var token = Request.Query["access_token"].FirstOrDefault();
      if (/* token validation fails */)
        return AuthenticateResult.Fail("Invalid Token");
      var ticket = new AuthenticationTicket(/* ... */);
      return AuthenticateResult.Success(ticket);
  }
}
```

AUTHENTICATION IN SIGNALR

- Then configuring the web application to use it:

```
builder.Services.AddAuthentication(
  options => {
    options.DefaultAuthenticateScheme = "Token";
    options.DefaultChallengeScheme = "Token";
  })
  .AddScheme<AuthenticationSchemeOptions,
             SimpleTokenHandler>("Token", null);
```

- Compare to Lecture 9: this is almost identical to the custom opaque token authentication for *TravelAgency*, but using query string instead of the *Authorization* header.

AUTHENTICATION ON THE CLIENT

SignalR clients must be configured for each hub to pass the access token as a *query string* parameter.

- For a TypeScript / JavaScript client:

```
const connection = new signalR.HubConnectionBuilder()
  .withUrl("/chatHub", {
    accessTokenFactory: () => authToken
  })
  .withAutomaticReconnect()
  .build();
```

- Where **`accessTokenFactory`** can be a function returning a **`string`** – or even a **`Promise<string>`** to support asynchronous execution.

AUTHENTICATION ON THE CLIENT

- For a .NET client:

```
var connection = new HubConnectionBuilder()
  .WithUrl("https://localhost:5001/chatHub", options => {
    options.AccessTokenProvider = () =>
      Task.FromResult("access token value");
  })
  .WithAutomaticReconnect()
  .Build();
```

- Where the **`AccessTokenProvider`** can be a custom Task<string>, loading the access token from a storage.

AUTHENTICATION ON THE CLIENT

- For a Java client:

```
HubConnection hubConnection = HubConnectionBuilder
  .create("https://localhost:5001/chatHub")
  .withAccessTokenProvider(Single.defer(() -> {
    return Single.just("access token value");
  }))
  .build();
```

- Uses RxJava's **`Single`** to provide async tokens. **`Single`** is reactive pattern, an observable emitting exactly one item.

HUB CONTEXT

For each hub, the base class provides the Context property with various information related to the connection:

- ConnectionId: gets the unique connection identifier (assigned by SignalR). There's one connection ID for each connection.
- UserIdentifier: gets the user identifier (case sensitive).
- User: gets the ClaimsPrincipal object associated with the user.
- Items: key/value collection that can be used to share data within the scope of the connection. Data will be persisted for the connection across different hub method invocations.

AUTHORIZATION IN SIGNALR

The **`[Authorize]`** attribute can be added to SignalR hubs or just individual methods for authorization with the ASP.NET Core middleware

- This ensures that only authenticated and authorized users can send or receive messages or perform sensitive actions.
- The user identifier for a connection can be accessed by the **`Context.UserIdentifier`** property in the hub.
  - By default, SignalR uses the **`ClaimTypes.NameIdentifier`** from the ClaimsPrincipal associated with the connection
- You can also check **`Context.User.Identity`** or use **`Context.User.IsInRole(…)`** manually inside hub methods

USER MANAGEMENT IN SIGNALR

A single user in SignalR can have multiple connections to an application

- E.g. a user could be connected on their desktop and their phone.
- Each device has a separate SignalR connection, but they're all associated with the same user.
- If a message is sent to the user, all connections associated with that user receive the message, e.g.:

```
await Clients.User("someuser")
              .SendAsync("ReceiveMessage", message);
```

- E.g. send a message to all clients of the current user:

```
if (Context.UserIdentifier != null)
  await Clients.User(Context.UserIdentifier)
               .SendAsync("ReceiveMessage", message);
```

CONNECTION MANAGEMENT IN SIGNALR

Connection and disconnection can be handled by overriding the **`OnConnectedAsync()`** and **`OnDisconnectedAsync()`** methods in a hub.

- Useful for managing groups, logging or cleaning up resources
- Examples:
  - Online/offline indicators for users
  - Real-time user list
  - Reassigning resources when someone leaves
- Both methods are guaranteed to be called, however the disconnection is not necessarily detected immediately on the server

CONNECTION MANAGEMENT IN SIGNALR

Client disconnection can occur for various reasons

- The client can gracefully close the SignalR connection, by calling the **`stop()`** / **`StopAsync()`** method on the hub connection object
  - The JS/TS SignalR client also detects the disconnection when the browser unloads the page and initiates graceful disconnection
- If the client closes abruptly (or the client cannot notify the server in time), the server detects the dropped connection after a timeout
  - When the client just lost connection, it will try to recover the connection, in which case the same connection will be kept
  - The server detects lost connections through transport-level timeouts (*keep-alive* + *ping-pong*)

SEND MESSAGES FROM OUTSIDE A HUB

The SignalR hub is the core abstraction for sending messages to clients connected to the SignalR server.

- It's also possible to send messages from other places in your app using the **`IHubContext`** service.
- An instance of **`IHubContext`** can be fetched via dependency injection and injected into an API controller, middleware, service class, etc.

```
public class HomeController : Controller {
  private readonly IHubContext<NotificationHub> _hubContext;
  public HomeController(IHubContext<NotificationHub> hubContext) {
  _hubContext = hubContext;
  }
}
```